

Trees & Heap Sort

1. Introduction to Trees

>> ◊ <<

Think of a tree as a hierarchy (like a family tree or a folder structure on your PC), not a straight line like an Array or Linked List.

Structure: It is non-linear. It has a Root (top node) and Children.

Binary Tree: A special tree where every node has at most 2 children (Left and Right).

Complete Binary Tree: A tree where every level is completely filled, except possibly the last level, which is filled from left to right.

Exam Tip: Heaps are always Complete Binary Trees. This allows us to store them easily in an array.

2. Heap Sort (Definition)

>> ◊ <<

Heap Sort is a comparison-based sorting technique based on the Binary Heap data structure.

How it works: It divides the input into a sorted and an unsorted region. It iteratively shrinks the unsorted region by extracting the largest element (Max Heap) or smallest element (Min Heap) and moving it to the sorted region.

Key Characteristic: It is an in-place algorithm (requires very little extra memory).

3. Min Heap and Max Heap (Tree Form)

>> ◊ <<

☞ This is the most likely concept to appear in "Short Question" sections.

Max Heap

Rule: The value of every Parent node must be greater than or equal to its children.

Root: The Root is always the Maximum element in the tree.

Use: Used to sort an array in Ascending order.

Min Heap

Rule: The value of every Parent node must be less than or equal to its children.

Root: The Root is always the Minimum element in the tree.

Use: Used to sort an array in Descending order.

4. Step-by-Step Solving (The "Heapify" Process)



Scenario: Sort array [4, 10, 3, 5, 1] using Max Heap.

Phase 1: Build the Max Heap



We look at the tree and fix violations (where child > parent) starting from the bottom-most non-leaf node.

(Array Indices: 4 is at 0, 10 is at 1, 3 is at 2, 5 is at 3, 1 is at 4)

- ↳ Initial Tree: Root is 4. Left child is 10, Right child is 3. 10 has children 5 and 1.
- ↳ Check index 1 (Value 10): Is $10 > 5$ and $10 > 1$? Yes. Yes. No swap needed.
- ↳ Check index 0 (Value 4): Left child is 10, Right child is 3. 10 is bigger than 4. SWAP.
- ↳ Re-check the swapped path: The new node at index 1 is now 4. Its children are 5 and 1. 5 is bigger than 4. SWAP.

Max Heap Built: [10, 5, 3, 4, 1]

Phase 2: Extract and Sort



Swap Root with Last: Swap 10 (max) with 1 (last).

Array: [1, 5, 3, 4, 10] 10 is now 'sorted' at the end. Ignore it.

Heapify Root: Fix the new root (1). Children of 1 are 5 and 3.

Max is 5. Swap 1 and 5.

Now check 1 at its new position. Child is 4.

Swap 1 and 4.

New Heap: [5, 4, 3, 1] | 10

Repeat: Swap Root (5) with last unsorted (1).

Array: [1, 4, 3, 5, 10] Heapify 1 again...

Final Sorted Array: [1, 3, 4, 5, 10]

5. Pseudocode

↳ Keep this simple. You generally need two functions: one to fix the heap (heapify) and one to run the sort.

```
// n is size of heap, i is index of root
FUNCTION heapify(arr, n, i)
    largest = i
    left = 2i + 1
    right = 2i + 2
    IF left < n AND arr[left] > arr[largest]
        largest = left
    IF right < n AND arr[right] > arr[largest]
        largest = right
    IF largest /= i
        swap(arr[i], arr[largest])
        heapify(arr, n, largest)
    // Recursively fix the affected sub-tree

FUNCTION heapSort(arr)
    n = length of arr
    // 1. Build Max Heap
    FOR i = n/2 - 1 down to 0
        heapify(arr, n, i)
    // 2. Extract elements one by one
    FOR i = n - 1 down to 0
        swap(arr[0], arr[i])
        heapify(arr, i, 0)
```

6. Time Complexity (Memorize This Table)

↳ Heap sort is very consistent. Unlike Quick Sort, it does not have a "bad" worst-case scenario.

Scenario	Complexity	Why?
Best Case	$O(n \log n)$	Even if sorted, we still build the heap and remove nodes.
Average Case	$O(n \log n)$	Standard performance.
Worst Case	$O(n \log n)$	Consistent performance regardless of data.
Space Complexity	$O(1)$	It sorts in-place (no new array needed).

Exam Cheat Sheet (Indices)

If you are solving this on paper with array index i:

Left Child: $2i + 1$

Right Child: $2i + 2$

Parent: $\lfloor (i - 1) / 2 \rfloor$

Greedy Algorithms & Kruskal's Algorithm

1. What is a Greedy Algorithm?

A Greedy algorithm builds a solution piece by piece, always choosing the next piece that offers the most immediate benefit.

“Notes by Humna Imran”

Goal: Used in these topics to find the Minimum Spanning Tree (MST) — connecting all nodes in a graph with the minimum total edge weight, without forming any cycles.

2. Kruskal's Algorithm “The Edge Picker”

Definition & Working

Concept: Sorts all edges from smallest to largest and adds them to the tree one by one.

Rule: Add the smallest edge unless it forms a cycle. If it forms a cycle, skip it.

Data Structure: Uses Union-Find (Disjoint Set) to check for cycles.

Step-by-Step

1. List all edges and sort them by weight (Low $\rightarrow$ High).
2. Pick the smallest edge. Connect the two nodes.
3. Pick the next smallest edge.

Check: Does adding this edge create a loop (cycle)?

- **Yes:** Discard it.
- **No:** Add it.

4. Repeat until you have $(V - 1)$ edges (where V is the number of vertices).

Kruskal's Algorithm

Pseudocode & Complexity

Pseudocode

FUNCTION Kruskals(Graph):

A = empty set (result)

SORT all edges by weight ascending

FOR each edge (u, v) in sorted list:

IF Find(u) != Find(v):

// If they are in different sets (no cycle)

Add edge (u, v) to A

Union(u, v)

// Merge the sets

RETURN A

Time Complexity



- Sorting Edges: $O(E \log E)$ or $O(E \log V)$
- Cycle Check (Union-Find): Nearly constant, effectively $O(1)$
- Total Complexity: $O(E \log E)$ or $O(E \log V)$

Prim's Algorithm

Pseudocode & Analysis



Pseudocode

```
FUNCTION Prims(Graph, startNode):  
  Create Min-Heap Q  
  Fill Q with all vertices  
  (set keys to infinity, startNode key to 0)  
  WHILE Q is not empty:  
    u = Extract-Min(Q)  
    // Get node with smallest cost  
    FOR each neighbor v of u:  
      IF v is in Q AND weight(u,v) < key[v]:  
        key[v] = weight(u,v)  
        parent[v] = u  
        Decrease-Key(Q, v, key[v])
```



Time Complexity

- Adjacency Matrix: $O(V^2)$
- Adjacency List + Binary Heap: $O(E \log V)$
- Adjacency List + Fibonacci Heap: $O(E + V \log V)$
(Best for theoretical exams)



Cheat Sheet: Kruskal's vs. Prim's

Feature	Kruskal's	Prim's
Approach	Focuses on Edges (connects isolated trees).	Focuses on Vertices (grows one single tree).
Best For	Sparse Graphs (fewer edges).	Dense Graphs (lots of edges).
Mechanic	Sorts edges first.	Uses a Priority Queue.

Graph Traversals (DFS & BFS)

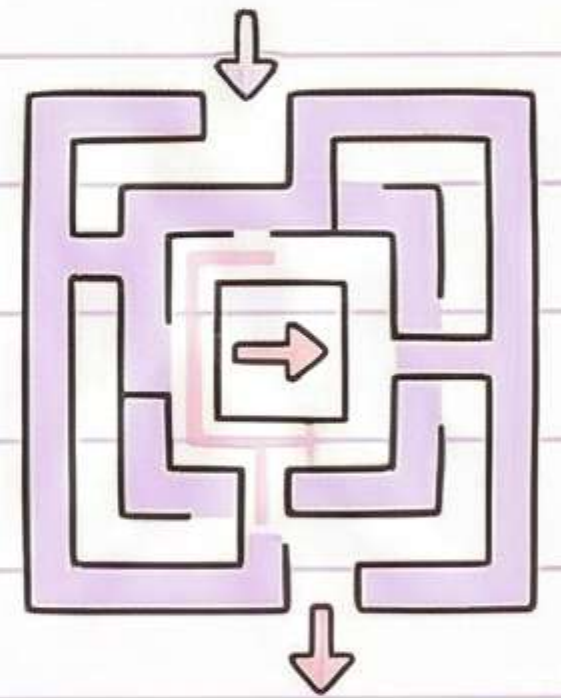
These are methods to visit every node in a graph.

A. DFS (Depth First Search) "The Maze Runner"

Goes as deep as possible, then backtracks.

Structure: Uses a Stack (or Recursion).

Working: Start at a node, choose a neighbor, and keep going forward until you hit a dead end. Then go back (backtrack) to the last junction and try a different path.



Pseudocode

```
FUNCTION DFS(node):  
    Mark node as visited  
    Print node  
    FOR each neighbor of node:  
        IF neighbor is NOT visited:  
            DFS(neighbor)  
            // Recursive call
```



Time Complexity

$$O(V + E)$$

Graph Traversals

(DFS & BFS)

B. BFS (Breadth First Search) "The Ripple Effect"

Explores neighbors layer by layer.



Structure: Uses a **Queue** (First In, First Out)

Working: Start at a node, visit all immediate neighbors first. Then visit the neighbors' neighbors.
(Like throwing a stone in water)



Pseudocode

```
FUNCTION BFS(startNode):
```

```
    Queue q
```

```
    Mark startNode as visited
```

```
    q.enqueue(startNode)
```

```
    WHILE q is not empty:
```

```
        curr = q.dequeue()
```

```
        Print curr
```

```
        FOR each neighbor of curr:
```

```
            IF neighbor NOT visited:
```

```
                Mark neighbor visited
```

```
                q.enqueue(neighbor)
```



Time Complexity

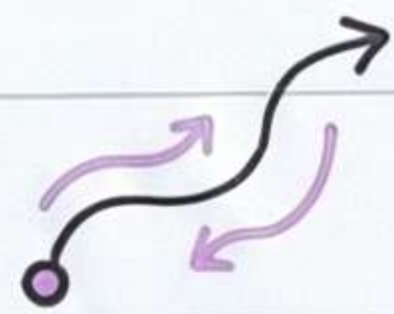
$O(V + E)$

Exam Tip: When to use which?

- Shortest Path in Unweighted Graph? → Use BFS
- Detecting a Cycle? → Use DFS
- Maze Solving / Puzzle? → Use DFS (usually)
- Social Network Friends (people within 2 hops)? → Use BFS

Dijkstra's Algorithm

Definition

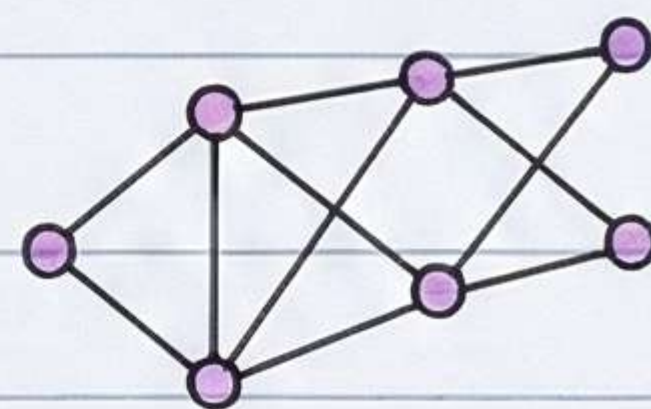


Dijkstra's algorithm is a Greedy Algorithm used to find the shortest path from a single source node to all other nodes in a graph with non-negative edge weights. It works like a "closest-first" search.

How it Works (Step-by-Step)

1. **Initialize** : Set distance of source node = 0.
All other nodes = ∞ (infinity).
2. **Select** : Pick the unvisited node with the smallest known distance.
3. **Relax** : Check neighboring nodes.
If new path (current distance + edge weight) is smaller $\rightarrow$ update distance.
4. **Repeat** : Mark current node as visited.
Repeat until all nodes are visited or destination is reached.

Time Complexity



- Using Array : $O(V^2)$

- Using Min-Priority Queue (Binary Heap) : $O((V + E) \log V)$

Where : V = Vertices.
 E = Edges

Pseudocode

```
function Dijkstra(Graph, source):
```

```
    dist[source] = 0
```

```
    create priority queue Q
```

```
    for each vertex v in Graph:
```

```
        if v  $\neq$  source
```

```
            dist[v] = infinity
```

```
            Q.add_with_priority(v, dist[v])
```

```
    while Q is not empty:
```

```
        u = Q.extract_min() // Node with smallest distance
```

```
        for each neighbor v of u:
```

```
            alt = dist[u] + length(u, v)
```

```
            if alt < dist[v]:
```

```
                dist[v] = alt
```

```
                Q.decrease_priority(v, alt)
```

```
    return dist
```

Key Limitation:

- Dijkstra fails if the graph has negative edge weights.

Bellman-Ford Algorithm

Definition

Bellman-Ford is a Dynamic Programming algorithm that finds the shortest path from a single source to all other nodes. Unlike Dijkstra, it can handle negative edge weights and can detect negative weight cycles.

How it Works (Step-by-Step)

- **Initialize:** Set distance to source as 0 and all others as infinity (∞).
- **Relax Edges:** Iterate $V-1$ times. In each iteration, go through every edge (u, v) with weight w . If $\text{dist}[u] + w < \text{dist}[v]$, update $\text{dist}[v] = \text{dist}[u] + w$.
- **Check for Negative Cycles:** Perform one more iteration. If any distance can still be reduced, it means there is a negative weight cycle in the graph.

Time Complexity

Average & Worst Case: $O(V \cdot E)$. It is slower than Dijkstra but more versatile.

Pseudocode

```
function BellmanFord(Graph, source):  
    dist[source] = 0  
    for each vertex  $v \neq \text{source}$ :  
        dist[v] = infinity  
  
    // Step 1: Relax all edges  $|V| - 1$  times  
    repeat  $|V| - 1$  times:  
        for each edge  $(u, v)$  with weight  $w$  in Graph:  
            if  $\text{dist}[u] + w < \text{dist}[v]$ :  
                 $\text{dist}[v] = \text{dist}[u] + w$   
  
    // Step 2: Check for negative weight cycles  
    for each edge  $(u, v)$  with weight  $w$  in Graph:  
        if  $\text{dist}[u] + w < \text{dist}[v]$ :  
            return "Graph contains a negative-weight cycle"  
  
    return dist
```

Feature	Dijkstra's Algorithm	Bellman-Ford Algorithm
Strategy	Greedy	Dynamic Programming
Negative Edges	✗ Cannot handle	✓ Can handle

Speed	Faster: $O(E \log V)$	Slower: $O(V \cdot E)$
-------	-----------------------	------------------------

Best Use	GPS, Network Routing	Finance (Arbitrage), Graphs with neg weights
----------	----------------------	--